

15. 上位速度計画の補間と warm start

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 list、dict、deque、NumPy 配列、pandas DataFrame	6
6.7 warm start は何を保存し、どう効くか	7
6.8 制御、状態、入力、モデル予測制御 MPC	7
7 関数・クラスを上から順に解説	7
7.1 L9 関数 _finite_vector	7
7.2 L16 関数 absolute_control_distances	8
7.3 L27 関数 shift_upper_policy_warm_start	9
7.4 L57 関数 load_upper_policy_csv	11
7.5 L81 関数 interpolate_upper_policy	12

8 処理の流れ	13
9 このファイルの位置づけ	13

1 対象

項目	内容
ファイル	mpc_solarcar/upper_policy.py
ソース SHA-256	8417d8498b74ae45328fb84745a5eb7cd940722eabb155b23e29baa3423d0252
種別	Python
区分	planner helper
役割	外部 speed policy CSV と前回解を絶対距離基準で現在メッシュへ補間し直す。
起動文脈	upper planner の初期値品質を決める補助。

2 このファイルを読む理由

外部 speed policy CSV と前回解を絶対距離基準で現在メッシュへ補間し直す。

- absolute_control_distances と shift_upper_policy_warm_start が重要。
- 相対距離ではなくルート絶対距離に揃える修正済み箇所。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_node.py
- scripts/solar_sim.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/upper_solver.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
import pandas as pd

def _finite_vector(values, *, name: str) -> np.ndarray:
    vector = np.asarray(values, dtype=float)
    if vector.ndim != 1 or len(vector) == 0 or not np.all(np.isfinite(vector)):
        raise ValueError(f"{name} must be a non-empty finite one-dimensional array")
    return vector
```

```
def absolute_control_distances(start_s_km: float, relative_control_s_km) -> np.ndarray:
    """Convert a horizon-relative control mesh to route-absolute distances."""
    start = float(start_s_km)
    if not np.isfinite(start):
        raise ValueError("start_s_km must be finite")
    relative = _finite_vector(relative_control_s_km, name="relative_control_s_km")
    if np.any(relative < -1.0e-9) or np.any(np.diff(relative) < -1.0e-9):
        raise ValueError("relative_control_s_km must be non-negative and non-decreasing")
    return start + relative

def shift_upper_policy_warm_start(
```

5 主要構造の読み取り

主要関数は `absolute_control_distances`, `shift_upper_policy_warm_start`, `load_upper_policy_csv`, `interpolate_upper_policy`。

5.1 主要クラス・関数

行	種別	名前	補足
9	function	<code>_finite_vector</code>	args: values
16	function	<code>absolute_control_distances</code>	args: start_s_km, relative_control_s_km
27	function	<code>shift_upper_policy_warm_start</code>	args: previous_control_s_km, previous_speeds_kmh, current_control_s_km
57	function	<code>load_upper_policy_csv</code>	args: path
81	function	<code>interpolate_upper_policy</code>	args: frame, control_s_km

5.2 ファイルを上から読んだときの定義順

行	内容
9	関数 <code>_finite_vector</code> を定義する。
16	関数 <code>absolute_control_distances</code> を定義する。
27	関数 <code>shift_upper_policy_warm_start</code> を定義する。
57	関数 <code>load_upper_policy_csv</code> を定義する。
81	関数 <code>interpolate_upper_policy</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future__ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。

3	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L57, L59。
5	<code>importnumpyasnp</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L9, L10, L11, L16, L19, L22, L34, L41, ...。
6	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L57, L60, L65, L66, L82, L90, L91。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘`x = f()`’では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

‘`float(...)`’、‘`int(...)`’、‘`bool(...)`’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘`None`’は値が無いことを示す単一のオブジェクト、‘`math.nan`’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘`is None`’と ‘`math.isfinite`’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 '(N,)」、候補集団なら '(population, N)「となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.7 warm start は何を保存し、どう効くか

warm start は前回または offline 探索で得た入力系列を、次の最適化の初期候補として渡すことである。答えを固定するのではなく、探索開始点と候補 library の一員を与える。

$$u_i^{0,\text{new}} = \text{interp} \left(s_i^{\text{new}}; s_j^{\text{old}}, u_j^{*,\text{old}} \right)$$

距離基準計画では、現在より後ろの旧制御点を捨て、新しい絶対距離制御点へ補間して shift する。初期状態や天候が変われば cost 評価は新条件で行われるので、warm start が不適切でも他 seed、CEM、安全 fallback が補う設計が必要である。

warm start の効き方は、局所法なら収束先と反復数、CEM なら初期 mean または候補 pool、receding horizon なら前回解の時間・距離 shift として現れる。どの位置に渡しているかを呼出引数まで追う。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- Rubinstein and Kroese: [The Cross-Entropy Method](#)

6.8 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L9 関数 _finite_vector

- 行範囲: L9–L13
- 所属: module 直下
- 定義: `_finite_vector(values, *, name:str) -> np.ndarray`

- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: ValueError, all, asarray, isfinite, len
- 戻り値の要点: vector
- 主なローカル代入名: vector
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: ValueError(f'{name} must be a non-empty finite one-dimensional array')
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. vector に np.asarray(values, dtype=float) の結果を代入する。
2. 条件 vector.ndim != 1 or len(vector) == 0 or (not np.all(np.isfinite(vector))) を判定し、真なら内部処理を行う。
3. ValueError(f'{name} must be a non-empty finite one-dimensional array') を送出する。
4. vector を返す。

代表コード断片

```
def _finite_vector(values, *, name: str) -> np.ndarray:
    vector = np.asarray(values, dtype=float)
    if vector.ndim != 1 or len(vector) == 0 or not np.all(np.isfinite(vector)):
        raise ValueError(f"{name} must be a non-empty finite one-dimensional array")
    return vector
```

7.2 L16 関数 absolute_control_distances

- 行範囲: L16–L24
- 所属: module 直下
- 定義: absolute_control_distances(start_s_km:float,relative_control_s_km)->np.ndarray
- docstring: Convert a horizon-relative control mesh to route-absolute distances.
- このブロックが直接呼ぶ主な関数・メソッド: ValueError, _finite_vector, any, diff, float, isfinite
- 戻り値の要点: start + relative
- 主なローカル代入名: relative, start
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: ValueError('relative_control_s_km must be non-negative and non-decreasing'), ValueError('start_s_km must be finite')
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. start に `float(start_s_km)` の結果を代入する。
2. 条件 `not np.isfinite(start)` を判定し、真なら内部処理を行う。
3. `ValueError('start_s_km must be finite')` を送出する。
4. relative に `_finite_vector(relative_control_s_km, name='relative_control_s_km')` の結果を代入する。
5. 条件 `np.any(relative < -1e-09) or np.any(np.diff(relative) < -1e-09)` を判定し、真なら内部処理を行う。
6. `ValueError('relative_control_s_km must be non-negative and non-decreasing')` を送出する。
7. `start + relative` を返す。

代表コード断片

```
def absolute_control_distances(start_s_km: float, relative_control_s_km) -> np.ndarray:
    """Convert a horizon-relative control mesh to route-absolute distances."""
    start = float(start_s_km)
    if not np.isfinite(start):
        raise ValueError("start_s_km must be finite")
    relative = _finite_vector(relative_control_s_km, name="relative_control_s_km")
    if np.any(relative < -1.0e-9) or np.any(np.diff(relative) < -1.0e-9):
        raise ValueError("relative_control_s_km must be non-negative and non-decreasing")
    return start + relative
```

7.3 L27 関数 `shift_upper_policy_warm_start`

- 行範囲: L27–L54
- 所属: module 直下
- 定 義: `shift_upper_policy_warm_start(previous_control_s_km, previous_speeds_kmh, current_control_s_km, *, minimum_speed_kmh: float, maximum_speed_kmh: float) -> np.ndarray`
- docstring: Shift a prior route-indexed policy onto the current absolute control mesh.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `_finite_vector`, `any`, `argsort`, `clip`, `diff`, `float`, `interp`, `len`, `unique`
- 戻り値の要点: `np.clip(shifted, float(minimum_speed_kmh), float(maximum_speed_kmh))`
- 主なローカル代入名: `current_s`, `keep`, `order`, `previous_s`, `previous_v`, `reverse_index`, `shifted`, `unique_s`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('current_control_s_km must be non-decreasing')`, `ValueError('previous control distances and speeds must have equal length')`
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `previous_s` に `_finite_vector(previous_control_s_kmh, name='previous_control_s_kmh')` の結果を代入する。
2. `previous_v` に `_finite_vector(previous_speeds_kmh, name='previous_speeds_kmh')` の結果を代入する。
3. `current_s` に `_finite_vector(current_control_s_kmh, name='current_control_s_kmh')` の結果を代入する。
4. 条件 `len(previous_s) != len(previous_v)` を判定し、真なら内部処理を行う。
5. `ValueError('previous control distances and speeds must have equal length')` を送出する。
6. 条件 `np.any(np.diff(current_s) < -1e-09)` を判定し、真なら内部処理を行う。
7. `ValueError('current_control_s_kmh must be non-decreasing')` を送出する。
8. `order` に `np.argsort(previous_s, kind='stable')` の結果を代入する。
9. `previous_s` に `previous_s[order]` の結果を代入する。
10. `previous_v` に `previous_v[order]` の結果を代入する。
11. `(unique_s, reverse_index)` に `np.unique(previous_s[::-1], return_index=True)` の結果を代入する。
12. `keep` に `len(previous_s) - 1 - reverse_index` の結果を代入する。
13. `keep` に `keep[np.argsort(unique_s)]` の結果を代入する。
14. `previous_s` に `previous_s[keep]` の結果を代入する。
15. `previous_v` に `previous_v[keep]` の結果を代入する。
16. `shifted` に `np.interp(current_s, previous_s, previous_v)` の結果を代入する。
17. `np.clip(shifted, float(minimum_speed_kmh), float(maximum_speed_kmh))` を返す。

代表コード断片

```
def shift_upper_policy_warm_start(
    previous_control_s_kmh,
    previous_speeds_kmh,
    current_control_s_kmh,
    *,
    minimum_speed_kmh: float,
    maximum_speed_kmh: float,
) -> np.ndarray:
    """Shift a prior route-indexed policy onto the current absolute control mesh."""
    previous_s = _finite_vector(previous_control_s_kmh, name="previous_control_s_kmh")
    previous_v = _finite_vector(previous_speeds_kmh, name="previous_speeds_kmh")
    current_s = _finite_vector(current_control_s_kmh, name="current_control_s_kmh")
    if len(previous_s) != len(previous_v):
        raise ValueError("previous control distances and speeds must have equal length")
    if np.any(np.diff(current_s) < -1.0e-9):
        raise ValueError("current_control_s_kmh must be non-decreasing")

    order = np.argsort(previous_s, kind="stable")
    previous_s = previous_s[order]
    previous_v = previous_v[order]
    unique_s, reverse_index = np.unique(previous_s[::-1], return_index=True)
    keep = len(previous_s) - 1 - reverse_index
```

```

keep = keep[np.argsort(unique_s)]
previous_s = previous_s[keep]
previous_v = previous_v[keep]

shifted = np.interp(current_s, previous_s, previous_v)
return np.clip(shifted, float(minimum_speed_kmh), float(maximum_speed_kmh))

```

7.4 L57 関数 load_upper_policy_csv

- 行範囲: L57–L78
- 所属: module 直下
- 定義: load_upper_policy_csv(path:str|Path)->pd.DataFrame
- docstring: Load a distance-indexed upper speed policy with strict schema checks.
- このブロックが直接呼ぶ主な関数・メソッド: Path, ValueError, copy, drop_duplicates, dropna, float, issubset, len, read_csv, replace, reset_index, sort_values
- 戻り値の要点: out
- 主なローカル代入名: frame, out, policy_path, required
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: ValueError(f'upper policy distance must increase: {policy_path}'), ValueError(f'upper policy must contain {sorted(required)}: {policy_path}'), ValueError(f'upper policy needs at least two finite distance points: {policy_path}')
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. policy_path に Path(path) の結果を代入する。
2. frame に pd.read_csv(policy_path) の結果を代入する。
3. required に {'s_kmh', 'v_kmh'} の結果を代入する。
4. 条件 not required.issubset(frame.columns) を判定し、真なら内部処理を行う。
5. ValueError(f'upper policy must contain {sorted(required)}: {policy_path}') を送出する。
6. out に frame.loc[:, ['s_kmh', 'v_kmh']].copy() の結果を代入する。
7. out['s_kmh'] に pd.to_numeric(out['s_kmh'], errors='coerce') の結果を代入する。
8. out['v_kmh'] に pd.to_numeric(out['v_kmh'], errors='coerce') の結果を代入する。
9. out に out.replace([np.inf, -np.inf], np.nan).dropna().sort_values('s_kmh').drop_duplicates('s_kmh', keep='last').reset_index() の結果を代入する。
10. 条件 len(out) < 2 を判定し、真なら内部処理を行う。

11. `ValueError(f'upper policy needs at least two finite distance points: {policy_path}')` を送出する。
12. 条件 `float(out['s_km'].iloc[-1]) <= float(out['s_km'].iloc[0])` を判定し、真なら内部処理を行う。
13. `ValueError(f'upper policy distance must increase: {policy_path}')` を送出する。
14. `out` を返す。

代表コード断片

```
def load_upper_policy_csv(path: str | Path) -> pd.DataFrame:
    """Load a distance-indexed upper speed policy with strict schema checks."""
    policy_path = Path(path)
    frame = pd.read_csv(policy_path)
    required = {"s_km", "v_kmh"}
    if not required.issubset(frame.columns):
        raise ValueError(f"upper policy must contain {sorted(required)}: {policy_path}")
    out = frame.loc[:, ["s_km", "v_kmh"]].copy()
    out["s_km"] = pd.to_numeric(out["s_km"], errors="coerce")
    out["v_kmh"] = pd.to_numeric(out["v_kmh"], errors="coerce")
    out = (
        out.replace([np.inf, -np.inf], np.nan)
        .dropna()
        .sort_values("s_km")
        .drop_duplicates("s_km", keep="last")
        .reset_index(drop=True)
    )
    if len(out) < 2:
        raise ValueError(f"upper policy needs at least two finite distance points: {policy_path}")
    if float(out["s_km"].iloc[-1]) <= float(out["s_km"].iloc[0]):
        raise ValueError(f"upper policy distance must increase: {policy_path}")
    return out
```

7.5 L81 関数 `interpolate_upper_policy`

- 行範囲: L81–L95
- 所属: module 直下
- 定義: `interpolate_upper_policy(frame:pd.DataFrame,control_s_km,*,minimum_speed_kmh:float,maximum_speed_kmh:float)->np.ndarray`
- docstring: Interpolate a learned full-course policy onto the current MPC control mesh.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `_finite_vector`, `all`, `clip`, `float`, `interp`, `isfinite`, `len`, `to_numeric`, `to_numpy`
- 戻り値の要点: `np.clip(interpolated, float(minimum_speed_kmh), float(maximum_speed_kmh))`
- 主なローカル代入名: `control_s`, `interpolated`, `source_s`, `source_v`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `ValueError('upper policy contains non-finite or insufficient points')`
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `control_s` に `_finite_vector(control_s_km, name='control_s_km')` の結果を代入する。
2. `source_s` に `pd.to_numeric(frame['s_km'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
3. `source_v` に `pd.to_numeric(frame['v_kmh'], errors='coerce').to_numpy(dtype=float)` の結果を代入する。
4. 条件 `len(source_s) < 2 or not np.all(np.isfinite(source_s)) or (not np.all(np.isfinite(source_v)))` を判定し、真なら内部処理を行う。
5. `ValueError('upper policy contains non-finite or insufficient points')` を送出する。
6. `interpolated` に `np.interp(control_s, source_s, source_v)` の結果を代入する。
7. `np.clip(interpolated, float(minimum_speed_kmh), float(maximum_speed_kmh))` を返す。

代表コード断片

```
def interpolate_upper_policy(
    frame: pd.DataFrame,
    control_s_km,
    *,
    minimum_speed_kmh: float,
    maximum_speed_kmh: float,
) -> np.ndarray:
    """Interpolate a learned full-course policy onto the current MPC control mesh."""
    control_s = _finite_vector(control_s_km, name="control_s_km")
    source_s = pd.to_numeric(frame["s_km"], errors="coerce").to_numpy(dtype=float)
    source_v = pd.to_numeric(frame["v_kmh"], errors="coerce").to_numpy(dtype=float)
    if len(source_s) < 2 or not np.all(np.isfinite(source_s)) or not np.all(np.isfinite(
        source_v)):
        raise ValueError("upper policy contains non-finite or insufficient points")
    interpolated = np.interp(control_s, source_s, source_v)
    return np.clip(interpolated, float(minimum_speed_kmh), float(maximum_speed_kmh))
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/upper_policy.py` を **planner helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。